

Approve once.

A governed logic database for
agentic decision support.

Deterministic, explainable inference — and human approval that attaches
to the logic rather than to each decision.

Group 3 — Human-Centred, Agentic, Decision-Support AI
Working prototype · 285 tests · three domains

The remedy everyone reaches for

“Put a human in the loop.”

- EU AI Act Article 14 requires effective human oversight of high-risk systems.
- In practice this means a reviewer approves or rejects **individual decisions**.

The remedy everyone reaches for

“Put a human in the loop.”

- EU AI Act Article 14 requires effective human oversight of high-risk systems.
- In practice this means a reviewer approves or rejects **individual decisions**.

That unit is the problem.

Two structural failures

It doesn't scale.

Cost is flat at one review per decision, however large the caseload grows.

It becomes ceremony.

Where the reviewer cannot verify the output, a signature transfers blame without improving the decision.

The failure is not of oversight.

It is a failure of **unit**.

The move

Take the decision logic **out of the model**
and put it in an **inspectable database**.

The move

Take the decision logic **out of the model**
and put it in an **inspectable database**.

Then review **rules**, not decisions.

The move

Take the decision logic **out of the model**
and put it in an **inspectable database**.

Then review **rules**, not decisions.

A rule reviewed once governs every future case its body matches.
The work amortises instead of repeating.

Two contributions, and the order matters

1. The logic database

Deterministic inference, three-valued evidence, exact bounds under missing facts, and a proof you can read.

2. Governance over it

The agent proposes, the engine infers, and approval attaches to rules rather than decisions.

Two contributions, and the order matters

1. The logic database

Deterministic inference, three-valued evidence, exact bounds under missing facts, and a proof you can read.

2. Governance over it

The agent proposes, the engine infers, and approval attaches to rules rather than decisions.

Approving a rule is a promise about *every future case*.

That promise is redeemable only if the engine makes it **deterministic** and **legible**.

Determinism is not a nice-to-have here

- Terminating fixpoint — no function symbols, so the derivable set is finite.
Structural, not a timeout.
- Each round matches a snapshot, so a rule's result cannot depend on *where in the round it ran*.
- Stratification re-checked on **every insertion** — because rules arrive from an agent over time.

Determinism is not a nice-to-have here

- Terminating fixpoint — no function symbols, so the derivable set is finite.
Structural, not a timeout.
- Each round matches a snapshot, so a rule's result cannot depend on *where in the round it ran*.
- Stratification re-checked on **every insertion** — because rules arrive from an agent over time.

A recursive rule the agent proposes cannot hang the system.
Insertion order cannot change the answer.

The defect we shipped twice

Standard stratified Datalog closes the world: a fact nobody recorded is *false*.

```
% guarantor(A) is UNKNOWN -- nobody recorded it  
mitigated(A) ← guarantor(A).  
decline(A) ← high_risk(A), not mitigated(A).
```

- mitigated is blocked by a missing fact $\Rightarrow$ reads as **false**
- so not mitigated *succeeds*
- and the applicant is declined on a guarantor **nobody ever asked about**

The defect we shipped twice

Standard stratified Datalog closes the world: a fact nobody recorded is *false*.

```
% guarantor(A) is UNKNOWN -- nobody recorded it  
mitigated(A) ← guarantor(A).  
decline(A) ← high_risk(A), not mitigated(A).
```

- mitigated is blocked by a missing fact $\Rightarrow$ reads as **false**
- so not mitigated *succeeds*
- and the applicant is declined on a guarantor **nobody ever asked about**

Uncertainty has to survive negation, or three-valued evidence
buys you nothing past the first not.

Exact bounds, in two evaluations instead of 2^k

With k unknown facts, the honest answer spans every way they could resolve. Naively that is 2^k evaluations.

- An unknown reachable only **positively** is minimised at false, maximised at true — set it directly.
- Only negatively? Set it the other way.
- Only unknowns reachable under **both** polarities need enumerating — in practice a handful.

Exact bounds, in two evaluations instead of 2^k

With k unknown facts, the honest answer spans every way they could resolve. Naively that is 2^k evaluations.

- An unknown reachable only **positively** is minimised at false, maximised at true — set it directly.
- Only negatively? Set it the other way.
- Only unknowns reachable under **both** polarities need enumerating — in practice a handful.

Checked against exhaustive enumeration on random programs *with* negation.
A shortcut nobody compares against the definition is a guess.

The engine's claims are tested, not asserted

Property	Tests
Bounds exact vs. brute-force enumeration	55
Termination and stratification	24
Determinism: same inputs, same answer	22
Proofs: provenance, assumed vs. established	7
Unknowns survive negation	5
Value of information	3
No circular self-support	1
Total	117 of 289

40% of the suite exists to make approval mean something.

The result

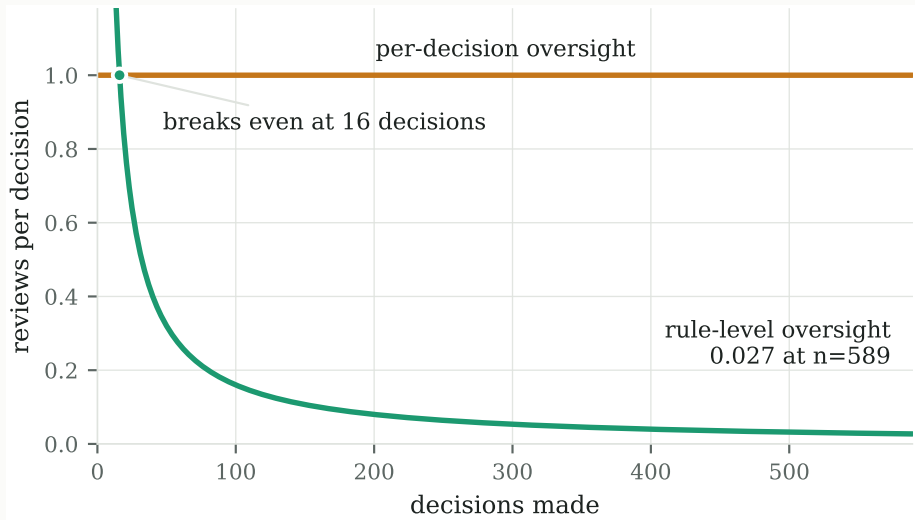
36.8

decisions governed per act of review

16 reviews · 589 decisions · 1,000 real credit applications

and the ratio is still climbing as the rule base matures

Oversight cost per decision



Reviews are paid up front — so it is *more* expensive at first, and breaks even at 16 decisions.

The commitment that makes it work

The agent **proposes**.
Only the engine **infers**.

- An LLM drafts candidate facts and rules. It never produces a decision.
- Inference is a terminating Datalog fixpoint — same inputs, same answer, every time.
- Nondeterminism is quarantined *upstream* of a human approval gate.

The commitment that makes it work

The agent **proposes**.
Only the engine **infers**.

- An LLM drafts candidate facts and rules. It never produces a decision.
- Inference is a terminating Datalog fixpoint — same inputs, same answer, every time.
- Nondeterminism is quarantined *upstream* of a human approval gate.

A hallucinated predicate never becomes a bad decision.
The vocabulary is closed; it becomes a discarded proposal nobody sees.

A rule carries its own governance

```
rule r0003 [p=0.80, origin=mined, status=approved,  
            by="reviewer-jd", support=214,  
            used=5, right="3/4"]  
high_risk(A) ← overdrawn(A), thin_savings(A).
```

- used — the amortisation count.
- right — a live tally, so strength is *auditable* rather than asserted.

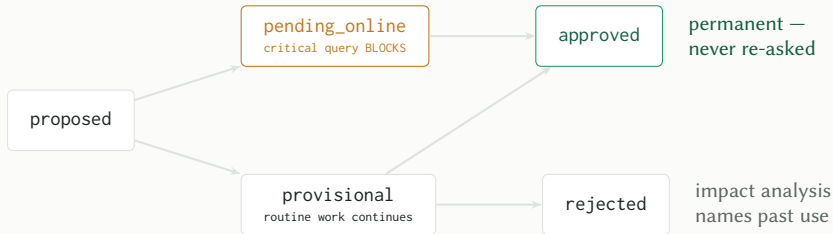
A rule carries its own governance

```
rule r0003 [p=0.80, origin=mined, status=approved,  
            by="reviewer-jd", support=214,  
            used=5, right="3/4"]  
    high_risk(A) ← overdrawn(A), thin_savings(A).
```

- used — the amortisation count.
- right — a live tally, so strength is *auditable* rather than asserted.

Identity is a **canonical** hash — variables renumbered, body sorted. Otherwise the same rule returns to the queue in different clothes and “approve once” means nothing.

Criticality routes review



Criticality attaches to the **query**. Approval attaches to the **rule**.

The invariant

A critical decision can only ever be produced by a derivation
in which **every rule is approved**.

```
BLOCKED critical query q00002 needs approval of: r0001, r0005.  
A critical decision may not rest on unreviewed logic.
```

Enforced in the engine, not by convention — and re-audited afterwards across every
decision. **Zero violations.**

What a decision looks like

Answer: $P = [0.00, 0.70]$

The interval is 0.70 wide because facts are missing. It is **not** a confidence interval over a model - it spans every way the unknowns could resolve.

[rule] app_gap should be declined $p=0.70$

[rule] by r0002: decline(A) $\leftarrow$ high_risk(A), **not** mitigated(A). $p=0.88$
(origin document; approved by policy-team; cites s7.1; right 70/191)

[fact] app_gap is overdrawn (source: application form, field 3)

[UNKNOWN] **not** app_gap has mitigating circumstances
(ASSUMED, **not** established)

Missing information, ranked by whether it would change anything:

app_gap has a guarantor [DECISIVE]
if true $\rightarrow 0.00$, if false $\rightarrow 0.70$

Three things that page does

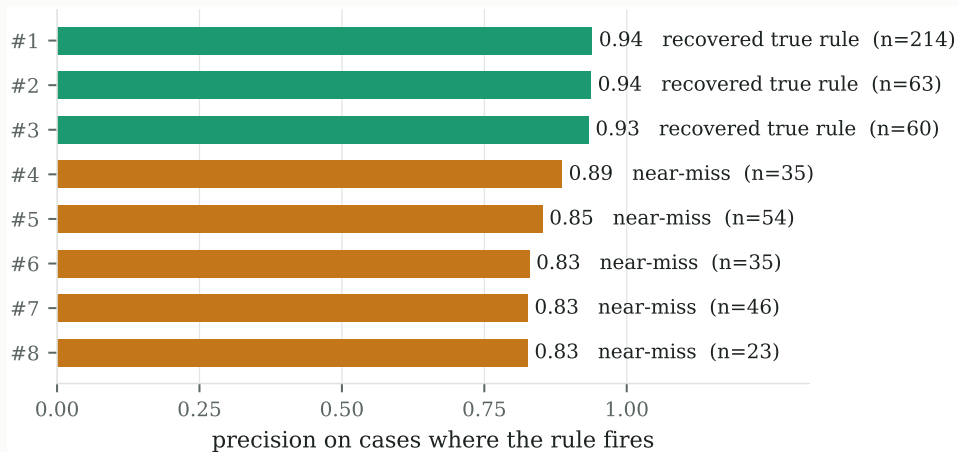
1. **Refuses to guess.** A missing fact produces an interval, not a number.
2. **Names the one question** that would settle it — and stays silent about unknowns that change nothing.
3. **Separates assumed from established**, so an interval endpoint cannot be misread as a finding.

Three things that page does

1. **Refuses to guess.** A missing fact produces an interval, not a number.
2. **Names the one question** that would settle it — and stays silent about unknowns that change nothing.
3. **Separates assumed from established**, so an interval endpoint cannot be misread as a finding.

We shipped (3) wrong twice. not mitigated(app) succeeded because nobody had
recorded a guarantor —
and an applicant was declined on an assumption no one had made.

Does the miner find the right rules?



Synthetic domain generated from a rule set that is then withheld — ground truth for **rules**, not predictions.

The near-misses are the point

Two of the five are wrong in ways a reviewer can *check*:

- One grants eligibility on a **recent award** — which policy s4.1 says should trigger *referral*.
- One reproduces the disability route with the **capital-limit guard dropped** — it would pay applicants over the savings limit.

Both sit at **83–85% precision**. Both look respectable.

The near-misses are the point

Two of the five are wrong in ways a reviewer can *check*:

- One grants eligibility on a **recent award** — which policy s4.1 says should trigger *referral*.
- One reproduces the disability route with the **capital-limit guard dropped** — it would pay applicants over the savings limit.

Both sit at **83–85% precision**. Both look respectable.

That is the case for a human reading rules.

Then the system found a hole in our own policy

We wrote a lending policy, extracted 8 cited rules — and it declined to decide **11% of cases**.

- s7.1 covers elevated risk *without* security
- s7.2 covers security *without* elevated risk
- **Neither covers an applicant with both.**

Then the system found a hole in our own policy

We wrote a lending policy, extracted 8 cited rules — and it declined to decide **11% of cases**.

- s7.1 covers elevated risk *without* security
- s7.2 covers security *without* elevated risk
- **Neither covers an applicant with both.**

We wrote it. We hadn't noticed.

A scoring model would have produced a confident number for every one of them.

We had already built the conventional version

A hundred simulated screening agents. 8,000 CVs. 1,000 hires.

Each agent may reject a confirmed mismatch or flag a CV for a person.

- It flags **5,621 of 8,000** CVs for human review — 4,012 of them cases where the requirements were *met*.
- Plus 237 sampled rejection audits.

We had already built the conventional version

A hundred simulated screening agents. 8,000 CVs. 1,000 hires.

Each agent may reject a confirmed mismatch or flag a CV for a person.

- It flags **5,621 of 8,000** CVs for human review — 4,012 of them cases where the requirements were *met*.
- Plus 237 sampled rejection audits.

5,858 human reviews to make 1,000 hires. 5.9 per hire.

And it cannot tell you when it is wrong

- Its interface says of agent rejections: “*each can be reopened*”.
- **2,142 of 2,379 rejections (90%) are never shown to anyone.**
- Rejections are correct *by construction* — there is no notion of a suitable applicant wrongly rejected.

And it cannot tell you when it is wrong

- Its interface says of agent rejections: “*each can be reopened*”.
- **2,142 of 2,379 rejections (90%) are never shown to anyone.**
- Rejections are correct *by construction* — there is no notion of a suitable applicant wrongly rejected.

An auditor has nothing to find.
Contestability is offered, and can never be exercised.

Same campaign, rules instead of cases

	Agent fleet	Rule base
One-off rule reviews	—	8
Human case reviews	5,858	2,558
Decided without a person	0	5,442
Screening logic inspectable	no	yes
Wrong rejections measurable	no	yes

Same campaign, rules instead of cases

	Agent fleet	Rule base
One-off rule reviews	—	8
Human case reviews	5,858	2,558
Decided without a person	0	5,442
Screening logic inspectable	no	yes
Wrong rejections measurable	no	yes

Against a withheld suitability label: **26.4%** of automatic rejections
are of genuinely suitable applicants.

Not a good number. But a number — and the audit queue now has something to find.

A disparity you can only see if you record it

age_band and sex are recorded in the data and are **not declared predicates** — so no rule can be written or mined over either.

age band	n	auto-rejected
under 25	2,713	38%
25–39	3,626	39%
40+	1,661	28%

A disparity you can only see if you record it

age_band and sex are recorded in the data and are **not declared predicates** — so no rule can be written or mined over either.

age band	n	auto-rejected
under 25	2,713	38%
25–39	3,626	39%
40+	1,661	28%

Over-40s are auto-rejected *less* — an age-correlated timeline signal routes them to a human instead.

A disparity toward more human review is still a disparity.

What we measured

Result	Value
Tests establishing engine properties	117 / 289
Bounds disagreeing with exhaustive enumeration	0
Decisions per act of review (credit)	36.8
Generating rules recovered exactly	3 / 3
Proposals resting on noise attributes	0 / 8
Cases on which written policy was silent	11%
Human case reviews vs. our agent fleet	−56%
Governance invariant violations	0

Every figure produced by scripts in the repository; both charts generated from live runs.

What we have *not* measured

No human has reviewed a rule in this system.

36.8 is arithmetic over a working prototype — not evidence that reviewers review rules *well*. If they cannot, the argument fails, and nothing here would currently tell us.

What we have *not* measured

No human has reviewed a rule in this system.

36.8 is arithmetic over a working prototype — not evidence that reviewers review rules *well*. If they cannot, the argument fails, and nothing here would currently tell us.

The experiment is unusually clean: show reviewers the eight proposals;
measure whether they approve the correct three.

Honest limitations

- **Credit assignment is crude.** Every rule in a derivation is marked right or wrong together.
- **Rejection flags but does not reverse.** It names affected decisions and stops — that judgement isn't ours to automate.
- **The accuracy figure is not a benchmark.** It scores an invented policy against real data.
- **The head-to-head is not like-for-like.** The baseline is our own prototype, and the two partition work differently.

The system detecting that its own policy is poorly fitted
is the machinery working, not a capability claim.

Datalog evaluation is from 1977. We borrow it unchanged.

What we built is an engine a reviewer can
trust,
and oversight that sits on the logic.

A reviewer reads one rule and governs every case it matches.
They can do that *because* the engine makes approval binding
and shows them what the rule did.